CHECK24 Fullstack Code Challenge

Symfony (Backend) & Vue.js (Frontend)

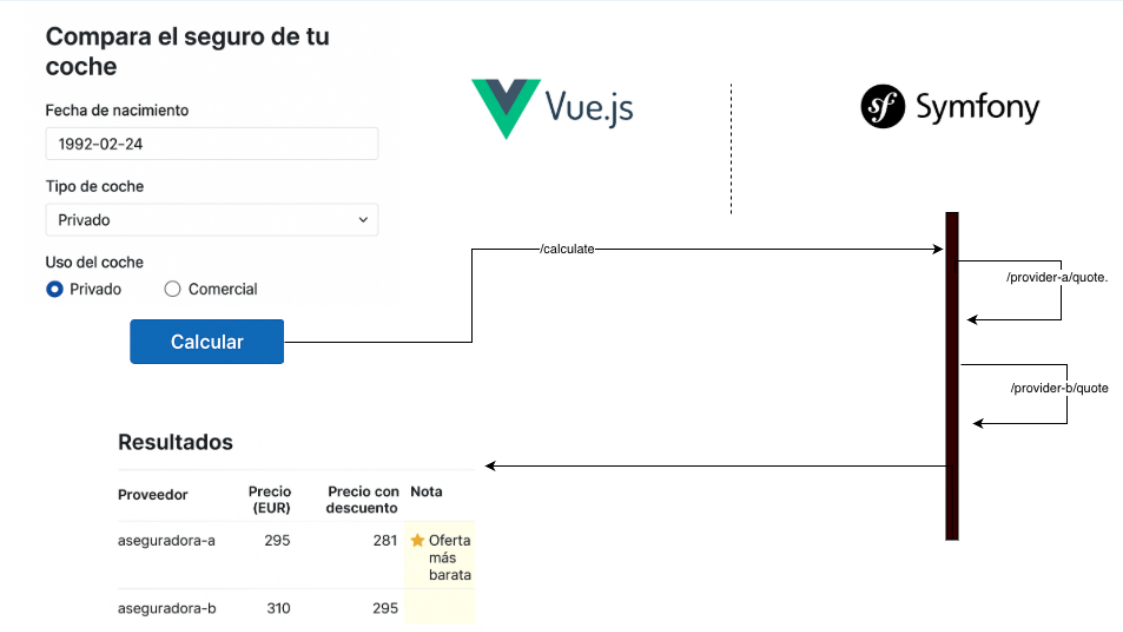
Background

Dear candidate,

Your task is to build a small web application that compares car insurance quotes from multiple providers. Each provider exposes a different API and pricing logic.

- The **backend** is a Symfony API. It receives the data the customer enters in the form, calls the provider APIs that simulate external insurers and returns a normalized, aggregated response.
- The **frontend** is a Vue.js single-page application (SPA) that collects user input and displays the comparison result.

You will implement both the backend and the frontend. Below you can find a design example and a use-case diagram to guide you:



Working Style

The use of LLMs while solving this task is not only allowed, but encouraged. If you use them, please include the implementation plan you worked from, together with any related planning artifacts created during the process, as part of your delivery package.

1 Backend Challenge

1.1 Scenario

Create a main use case with the following logic:

1) Implement mock APIs for two providers (simulating real insurers).
2) Implement a comparison endpoint that calls these providers, aggregates and returns their offers.
3) Apply a 5% discount if a campaign is active.

The focus of the task is on the structure, readability and correctness of your implementation.

1.2 Provider Simulation

Implement two endpoints that simulate car insurance providers. Each should receive a request with driver and car data and return a price offer.

Provider A — JSON API

This is a fast connection, the provider should take about 2 seconds to respond - add a sleep(2) in your code to simulate this. But in 10% of the calls, it should return an error 500, because the provider is unreliable.

Endpoint: `POST /provider-a/quote`

Request (JSON):

```
{
  "driver_age": 30,
  "car_form": "sedan",
  "car_use": "private"
}
```

Pricing Logic:

- Base price: 217€
- Age adjust:

  - 18-24 Add 70€
  - 25-55 Add 0€
  - 56- Add 90€
- Vehicle adjust:
  - SUV Add 100€
  - Compact Add 10€
- Add 15% if car is in commercial use.

Response (JSON):

```
{
  "price": "295 EUR"
}
```

This provider has only 2 car forms, `Turismo` and `Compacto` are both mapped to `compact`.

Provider B — XML API

This is a slow connection, add a sleep(5) to simulate calculation on the provider side. In reality, some providers take up to 1 min to answer, but we do not want to make it complicated for you.

The provider is more solid than provider A, but in 1% of the cases, this call takes 1 minute.

Endpoint: `POST /provider-b/quote`

Request (XML):

```
<SolicitudCotizacion>
    <EdadConductor>30</EdadConductor>
    <TipoCoche>turismo</TipoCoche>
    <UsoCoche>privado</UsoCoche>
    <ConductorOcasional>NO</ConductorOcasional>
</SolicitudCotizacion>
```

Pricing Logic:

- Base price: 250€
- Age adjust:
  - 18-29 Add 50€
  - 30-59 Add 20€
  - 60- Add 100€
- Vehicle adjust:
  - Turismo Add 30€
  - SUV Add 200€
  - Compacto Add 0€

Response (XML):

```
<RespuestaCotizacion>
    <Precio>310.0</Precio>
    <Moneda>EUR</Moneda>
</RespuestaCotizacion>
```

1.3 Calculation Endpoint

Implement a main calculation endpoint:

- **Endpoint:** `POST /calculate`
- **Request:** same fields as the provider APIs.

```
{
  "driver_age": <value>,
  "car_type": <value>,
  "car_use": <value>
}
```

Behaviour:

- Calls both provider endpoints over HTTP(S).
- Collects and normalizes their responses.
- Applies an active campaign discount if applicable.
- Returns the offers sorted by price (ascending).
- In case a connection takes >10 secs, stop the request and do not include the provider to ensure good customer experience.
- In case one or both of the connections have an issue, do not return prices for the provider.

1.4 Campaign rule

Decide how to enable/disable the campaign:

- Environment variable
- Configuration file
- Feature flag configurable over DB or backend tooling

1.5 Backend Deliverables

- Provider A & B endpoints and `/calculate` endpoint
- Campaign logic

- Automated tests covering at least:
  - Provider price calculations
  - Comparison and sorting logic
  - Campaign discount application

2 Frontend Challenge (Vue.js)

Create a Vue.js application that interacts with the Symfony backend API.

2.1 Functional Requirements

2.1.1 Input Form

Implement a form where the user can enter the data required by the backend:

- `driver_birthday` (date field)
- `car_type` (select field) with values: `Turismo`, `SUV` or `Compacto`
- `car_use` (radio box) with values: `Privado` o `Commercial`

Client-side validation:

- `driver_birthday` is required and must be a valid date with reasonable limits.
- `car_type` and `car_use` are required.

Submitting the form sends a `POST` request to `/calculate` with the following JSON payload:

```
{
  "driver_birthday": "<date>",
  "car_type": "<string>",
  "car_use": "<string>"
}
```

2. State Persistence Requirement

The form should be prefilled when the page is reloaded, but *not* when the browser tab or window has been closed.

Concretely:

- If the user reloads the page, the form must keep its values.
- If the user closes the browser tab/window and later opens the application again, the form must be empty (initial state).

3. Campaign Rule

If a campaign is active, CHECK24 covers 5% of the insurance price (i.e. the customer pays 95% of the original price).

The quote returned must content the discount applied in the result representation showing the price.

4. Backend Call

- Show a loading state while the request is in progress; at least the submit button must not be accessible.
- Handle error cases (network error, HTTP 4xx/5xx) and display a user-friendly error message.

5. Display of Quotes

After a successful request, display the returned quotes in a table or list, for example:

Proveedor	Precio (EUR)	Precio con descuento (EUR)	Nota
provider-a	295	281	Precio más barato*
provider-b	310	295	

Requirements:

- The quotes must be shown sorted by price ascending (you may rely on the backend or re-sort on the frontend).
- Visually highlight the cheapest quote (e.g. bold text, badge, icon).
- If due to problems on provider side (See above, 1% and 10% rules) both providers do not return prices, show a message... "No hay ofertas disponibles.".
- There should be a toggle to order by price asc/desc. Do not use a library to display the table, implement it yourself.

Think about how to display the applied campaign to the user so he realises he has special prices.

2.2 Technical Requirements

- Use Vue.js.
- Build a clear component structure.
- Do not rely on heavy visual styling; focus on usability and clarity.

2.3 Frontend Deliverables

- Input form component.
- Quote list component.
- Error and loading handling.
- Table sorting.

2.4 Submission Requirements

Submission Checklist

The use of LLMs while solving this task is not only allowed, but encouraged. If you use them, please include the implementation plan you worked from, together with any related planning artifacts created during the process, as part of your delivery package.

- Submit the full source code of your solution.
- In addition to the code, submit the **plan documents** you used during implementation or code generation.
- These plan documents should make your approach understandable for the reviewer (for example: implementation plans, task breakdowns or other planning artifacts used to guide the work).

Evaluation Criteria

We will look at:
Architecture

- Separation of concerns (controllers/services in Symfony, components/services in Vue).
- Clear, consistent folder structure. Imagine that this will be a large project - build your code accordingly to ensure you will still be able to understand the structure if hundreds a features have been added later.
- Our real connections are much more complex, this is stripped down for your benefit. Please treat this as if the requests actually have >60 fields, and build your code accordingly. You are free to structure controllers/services as you like, but keep the code modular and readable.
- RESTful API

Code Quality

- Readable, maintainable, and testable code.
- Sensible naming and comments only where necessary.
- Ensure code quality with tooling

Correctness

- Correct implementation of the provider pricing logic.
- Correct aggregation, sorting, and discount handling.
- Correct persistence of the form values.

Frontend UX

- Form validation and feedback (loading, errors).
- Clear and readable display of quotes.

Testing

- Automated tests for backend logic.
- Automated Frontend tests.

Senior Profiles only

Only for frontend seniors profiles

- Divide the input page in 3 pages with one question each and a `Back`/`Continue` navigation.
- Add lateral transitions when navigating between the 3 pages as in an iOS app.
- Make the frontend responsive for use in mobile/table/desktop.

Only for backend senior profiles

- Fetch provider quotes in parallel.
- OpenAPI descriptions
- Robust error handling for unavailable providers.
- Logging or minimal monitoring.
- A third provider with a different format (e.g. CSV or different JSON structure).
- Simple Docker setup for easy startup.